Version 2.0.0 — SpinGlassPEPS.jl: Tensor-network package for Ising-like optimization on quasi-two-dimensional graphs

Tomasz Śmierzchalski[a], Anna Maria Dziubyna[b,c], Konrad Jałowiecki[a], Zakaria Mzaouali[a], Łukasz Pawela[a,*], Bartłomiej Gardas[a], Marek M. Rams[b]

[a] *Institute of Theoretical and Applied Informatics, Polish Academy of Sciences, Bałtycka 5, 44-100 Gliwice, Poland*
[b] *Jagiellonian University, Institute of Theoretical Physics, Łojasiewicza 11, 30-348 Kraków, Poland*
[c] *Jagiellonian University, Doctoral School of Exact and Natural Sciences, Łojasiewicza 11, 30-348 Kraków, Poland*

Abstract

This update consolidates the four separately registered packages of `SpinGlassPEPS.jl` into one installable package with a curated public API, and adds capabilities the original release lacked. Truncating factorizations now report the weight they discard, so a heuristic contraction can state how much of the distribution it threw away without recourse to an exact oracle, and an inverse-temperature ladder with warm-started boundary matrix product states makes choosing β empirical rather than a guess. Profiling shows the solver is host-bound at every size measured, with garbage collection consuming 18–33% of a solve; addressing that yields $\approx$1.15$\times$ on each device. We also establish where GPU execution begins to pay, which is later than the original article implies.

Keywords: spin glasses, tensor networks, truncation error, QUBO, Ising model, reproducibility

Refers to

T. Śmierzchalski, A. M. Dziubyna, K. Jałowiecki, Z. Mzaouali, Ł. Pawela, B. Gardas, M. M. Rams, *SpinGlassPEPS.jl: Tensor-network package for*

*Corresponding author.
Email address: `lpawela@iitis.pl` (Łukasz Pawela)

Ising-like optimization on quasi-two-dimensional graphs, SoftwareX **31** (2025) 102257. https://doi.org/10.1016/j.softx.2025.102257

Metadata

Nr	Code metadata description	Metadata
C1	Current code version	v2.0.0 [tag and register before submission; no v2.0.0 tag exists yet]
C2	Permanent link to code/repository used for this code version	https://github.com/euro-hpc-pl/SpinGlassPEPS.jl
C3	Legal code license	Apache License 2.0
C4	Code versioning system used	git
C5	Software code languages, tools and services used	Julia; CUDA (optional, for GPU execution)
C6	Compilation requirements, operating environments and dependencies	Julia $\geq$ 1.11; CUDA.jl 5, cuTENSOR 2, TensorOperations.jl 5, LowRankApprox.jl, Graphs.jl, MetaGraphs.jl. A CUDA-capable GPU is optional; the CPU path is exercised by continuous integration. HDF5 is an optional package extension
C7	If available, link to developer documentation/manual	https://euro-hpc-pl.github.io/SpinGlassPEPS.jl/dev/
C8	Support email for questions	lpawela@iitis.pl

Description of the software-update

Motivation. The original publication [1] described `SpinGlassPEPS.jl` as separately registered Julia [2] packages in version lockstep, whose interface had since drifted: renames shipped without deprecation paths, so the listings printed in the 2025 article no longer ran. More consequentially, the solver reported no measure of how much its heuristic contraction discarded, so a result could be validated only against an exact oracle — as the original benchmark had to do, certifying ground states with CPLEX. Algorithms and an extensive benchmark campaign are described separately [3]; this version is archived [4].

One package, one interface. The four implementation packages (`SpinGlassTensors`, `SpinGlassNetworks`, `SpinGlassExhaustive`, `SpinGlassEngine`) are now internal modules of a single installable `SpinGlassPEPS`, so `pkg> add SpinGlassPEPS`

installs the complete solver. `using SpinGlassPEPS` brings a curated surface of roughly eighty documented symbols instead of re-exporting some 240; lower-level kernels remain reachable through the submodules, and deprecation shims restore the API forms printed in the original article, so its listings execute again.

Two internal changes have external consequences. The clustered Hamiltonian is now a concrete `PottsHamiltonian` type with typed fields read through accessors, rather than a `MetaGraphs` property bag consulted dynamically inside the search loop. And the process-global memoization that cached boundary matrix product states (MPS), MPO layers and environments is replaced by a contraction cache owned by each contractor, with explicit per-row eviction — which is what makes any form of concurrent solving possible: the previous design was thread-unsafe by construction and had to be cleared by manual calls scattered through the search.

The update also repairs defects that silently affected results rather than mere convenience: `corner_matrix` threw for every `SiteTensor` input and returned permuted trailing dimensions for `VirtualTensor`, and the exhaustive GPU search launched too few blocks, truncating the reported spectrum for problems larger than about nine spins. Finally, the benchmark instances at the size used by the original article's figures are now shipped with the package; the published listing resolved them from a directory that did not contain them.

Contraction error control. Every truncating factorization now records the relative weight it discards, $\varepsilon_i = \|\Sigma_{\text{dropped}}\|^2/\|\Sigma\|^2$, into an accumulator scoped to the running task, so concurrent solves cannot pool statistics. A solve reports $\sum_i \varepsilon_i$ — the leading term of the accumulated fidelity loss of a boundary-MPS sweep — with $\max_i \varepsilon_i$, the number of truncations forced by the bond-dimension bound rather than the singular-value tolerance, and retained-versus-offered dimensions. That last count answers a question users could not previously ask. On a 128-spin instance, bond dimension $D = 4$ gives $\sum \varepsilon = 3.1 \times 10^{-4}$ with all 18 truncations forced by the bond bound, retaining 108 of 307 singular values; at $D = 32$ the same solve gives 5.6×10^{-14} with no truncation bond-limited, retaining 192 of 592. A bond-limited count of zero means the bond dimension was never the binding constraint, so raising it cannot change the answer — previously a matter of guesswork.

Across the eight-transformation protocol the solver additionally reports the spread of energies and how many transformations reached the best one. Eight independent contraction orders agreeing is oracle-free evidence; a spread that is a sizeable fraction of the energy scale says the result should not be believed however good the best energy looks.

An inverse-temperature ladder. β is the most consequential parameter and was the least guidable — too small and the conditional probabilities the search branches on say little about the ground state — yet the original interface took a scalar whose optimum its own documentation conceded to be instance-dependent. `beta_ladder` walks an increasing schedule in which each rung reuses the previous rung's boundary MPS to start variational compression, instead of forming $W\psi$ exactly and truncating it. Where boundary-MPS construction dominates (2048-spin instance) this is ≈15% faster per warm-started rung (48.2 → 41.7 s, 49.3 → 41.2 s) for identical energies; where the search dominates it falls to 4–6%.

Two caveats bound how far the error measure can be pushed. Warm starting and the discarded-weight measure are incompatible: a warm start optimizes *within* a fixed bond dimension, never truncates, and so reports $\sum\varepsilon \approx 0$ whatever its accuracy, its error being a variational gap the log cannot see; audits must use cold builds, and the package warns. Less obviously, $\sum\varepsilon$ does not rank solution quality. In Figure 1(c) the energy error falls monotonically with β — reaching the best energy found in 9 of 10 instances at $\beta = 6$ — while $\sum\varepsilon$ peaks at $\beta = 4$ and then falls, because a sharply peaked Boltzmann distribution carries less entanglement and truncates more easily. The best βs thus had among the *lowest* discarded weight, and a guard on $\sum\varepsilon$ alone would have rejected them. It bounds what the contraction discarded, not whether the search found a good state.

A further consequence is visible in Figure 1(d). Search breadth does not buy diversity: at $M = 1024$ forty-nine of the fifty runs returned a single valley, the states differing from the best by 10–70 spins out of 2500 where a diversity count would demand 625; the lone exception (β=8, instance 1) is a degraded contraction, not a genuine second optimum (Figure 1 caption). What the metric of the original article scores as two solutions is the exact global spin flip — these instances carry no local fields, so $E(\sigma) = E(-\sigma)$ identically, and the mirror sits at distance N for free. Contraction order is what moves the solver between valleys: one state from each of the eight transformations gives two or three mutually distant solutions on four of five instances, with pairwise distances reaching 1249 of a possible 1250. This is an argument for sweeping transformations that does not appeal to energy at all, and it holds even where the transformations agree on the energy to several digits.

Concurrent transformation sweeps. The standard protocol solves each instance once per lattice transformation, a loop that previously lived in user code. `sweep_transformations` owns it, seeds each transformation deterministically (the `Zipper` strategy draws a random sketch, so results

Table 1: Speed-up of the concurrent sweep over the serial eight-transformation loop: median of per-round paired ratios, interleaved arms, full reclaim before every timed section. CPU 20 cores / BLAS 12 threads, 5 rounds; GPU RTX 5080, 7 rounds.

Case	Device	c=1	c=2	c=4	c=8
Chimera 3×4×3, D=16	CPU	0.52	0.87	1.32	**1.76**
Chimera 128-spin, D=32, `Zipper`	CPU	0.97	1.15	1.37	**1.39**
Chimera 3×4×3, D=16	GPU	0.68	0.92	0.80	—
Chimera 128-spin, D=32, `Zipper`	GPU	0.94	0.88	0.89	—

otherwise depend on task scheduling), isolates failures, and reports the diagnostics above. Since the limit on fanning out is device memory and depends on the instance rather than the machine, concurrency is gated by a byte budget: a calibration solve measures peak usage, and each admitted solve carries its reservation as a task-scoped value so kernel batches are sized against *its* slice rather than the shared pool. Without that last step the reservation would be advisory only: every concurrent solve would measure the same free pool and batch as though it owned all of it.

What this buys depends entirely on the device (Table 1). On CPU the sweep scales monotonically, reaching 1.76× and 1.39× at eight-way on the two cases; on a single GPU no concurrency level beats the serial loop. The solves do overlap — 5.4× at eight-way — but per-solve time degrades at the same rate, so total work is conserved and the driver's own overhead makes it a net loss. The default is therefore device-dependent.

One methodological caution, because it cost us a wrong result before it was caught: timing the serial arm immediately after a concurrent warm-up leaves the CUDA memory pool in a state that inflated our baseline by up to 2.7×, producing an apparent 1.68× speed-up where there is none. Interleaving the arms, reclaiming before every timed section, and reporting paired per-round ratios is necessary for any performance claim about this solver.

When the GPU is worth using. The original article recommends the GPU for its larger examples without qualification. Solving identical configurations on each device (Figure 2, Table 2) shows that holds only above a threshold easy to fall below: the host is ≈45× faster at 36 spins and still 1.5× faster at 2048 spins and bond 16, losing only once both instance and bond dimension are large — 2048 spins at bond 32, by 3% dense and 17% sparse. Energies agree in every cell. Exploratory work therefore belongs on the CPU; the D-Wave-scale sparse regime is where the device earns its place.

Table 2: Data behind Figure 2: host versus device on identical configurations, as CPU/GPU wall-clock ratio (below 1 means the CPU is faster). 'Zipper', 'SquareSingleNode', β=3, `max_states`=128 throughout; separate alternating processes.

Spins	D=8	D=16	D=32	D=64
36 (dense)	0.02	0.02	0.02	0.02
128 (dense)	0.15	0.38	0.48	0.56
128 (sparse)	—	—	—	0.77
2048 (dense)	0.43	0.68	**1.03**	—
2048 (sparse)	—	0.71	**1.17**	—

Where the time actually goes. Profiling explains both tables and redirects the obvious optimizations. The device is never the constraint: GPU utilization is 5.9% on a 128-spin solve and 12.8% at 2048 spins, while host-side CUDA API calls account for only 21–25% of wall time. The remaining two thirds is host-side Julia work, of which garbage collection alone is 18% (GPU) to 33% (CPU) of a 2048-spin solve. The solver is host-bound at every size we measured — which is also why the two devices tie at the crossover: both are waiting on the same bookkeeping, and the device's tensor work is nearly free but removes none of it.

An idle device invites kernel-level batching, and we did not pursue it for a reason worth recording. Eliminating the entire CUDA API share would cap at $\approx$1.3×, whereas the host-side majority was reachable directly. Two allocation changes follow (Figure 3). The branched-configuration set is now backed by a single matrix rather than one vector per state, which on a 2048-spin bond-32 solve is 1.16× end to end with garbage collection down 36% — while allocated *bytes* fall only 4%, since collection cost tracks the number of live objects rather than their volume, and the previous form created tens of thousands of small vectors per call. Separately, the multi-tensor contraction temporaries were moved off the collected heap onto explicit allocation, worth 1.25× on CPU with allocated bytes down by two thirds; in isolation that allocator is $\approx$7% *slower* per call, so the gain is entirely the collection it avoids and had to be measured in a full solve rather than a microbenchmark.

Impact, limitations and outlook. Contraction error is now an observable rather than an article of faith, letting a practitioner distinguish a converged run from a broken one on a problem where no exact answer is available — the regime the method exists for — provided it is read as a statement about the contraction and not about solution quality. The consolidation and the deprecation shims make the published interface reproducible, and shipping the benchmark instances closes a gap in that reproducibility

that predates this update.

The limitations are equally plain. The sweep is a genuine speed-up on CPU but not on a single GPU, for reasons we measured rather than assumed. Warm starting and the discarded-weight measure cannot be combined, and warm starting currently covers only the bottom boundary MPS, so layouts that build the top boundary do not benefit on those rows. The error measure answers a narrower question than one might hope: it bounds what the contraction discarded, and our own attempt to use it to select β failed, which is why the package now documents it as a diagnostic rather than a criterion.

The remaining headroom is host-side rather than in the tensor kernels. The branch-and-bound bookkeeping still dominates allocation even after the change above, and the sparse contraction path — which is where the device wins, per Table 2 — has not received the allocator treatment the dense one now has. Both are more promising than the batching the idle device superficially suggests.

Acknowledgements

[Carry over funding acknowledgements from the original publication and add any new sources.]

References

[1] T. Śmierzchalski, A. M. Dziubyna, K. Jałowiecki, Z. Mzaouali, Ł. Pawela, B. Gardas, M. M. Rams, SpinGlassPEPS.jl: Tensor-network package for Ising-like optimization on quasi-two-dimensional graphs, SoftwareX 31 (2025) 102257. doi:10.1016/j.softx.2025.102257.

[2] J. Bezanson, A. Edelman, S. Karpinski, V. B. Shah, Julia: A fresh approach to numerical computing, SIAM Review 59 (1) (2017) 65–98. doi:10.1137/141000671.

[3] A. M. Dziubyna, T. Śmierzchalski, B. Gardas, M. M. Rams, M. Mohseni, Limitations of tensor network approaches for optimization and sampling: A comparison against quantum and classical Ising machines, arXiv preprint (2024). arXiv:2411.16431.

[4] Ł. Pawela, et al., SpinGlassPEPS.jl, archived release. doi:10.5281/zenodo.14627393.

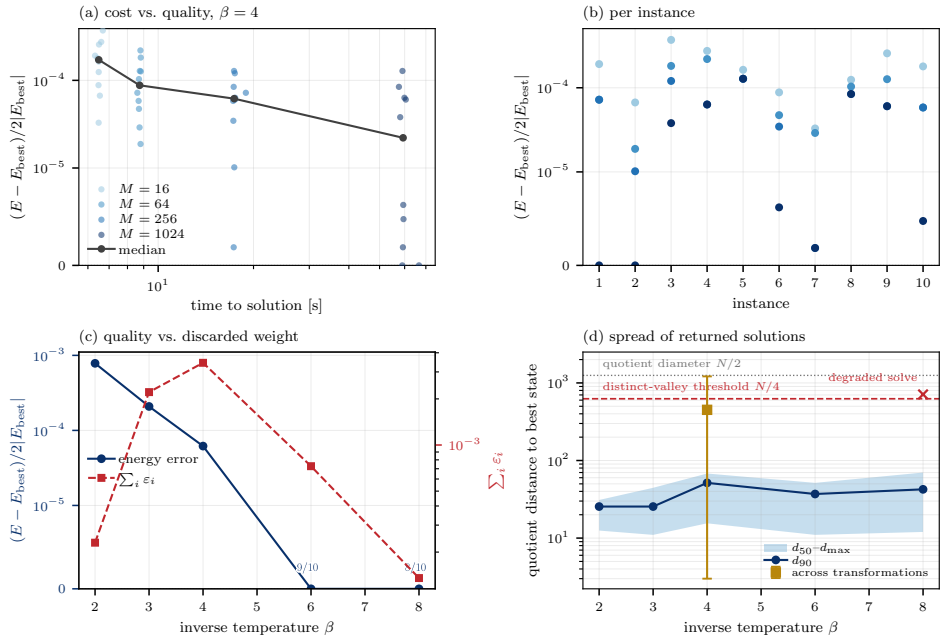

Figure 1: Ten 2500-spin (50×50) square-lattice instances at bond dimension 8, on CPU. (a) Time to solution against relative energy error as the search breadth $M =$ `max_states` varies over 16–1024; points are individual instances, the line their median. (b) The same runs resolved per instance. (c) Median energy error and median discarded weight $\sum_i \varepsilon_i$ against β; annotations count instances reaching the best energy found. E_{best} is the lowest energy over all settings here, not a certified optimum, so a zero error means "best found", not "optimal". The two curves in (c) diverge: quality improves monotonically with β while discarded weight peaks at $\beta = 4$ and then falls. (d) How far apart the returned solutions actually are, at $M = 1024$. These instances carry no local fields, so $E(\sigma) = E(-\sigma)$ exactly and every state has a degenerate mirror at Hamming distance N; distances are therefore quotient distances $\min(d, N - d)$, on a space of diameter $N/2$. Band and line are medians over the ten instances of the 50th, 90th and 100th percentile of the distance from each returned state to the best one. The whole returned set is a single tight cluster, one to two orders of magnitude below the $N/4$ threshold a diversity count would apply. The one point clearing it ($\beta = 8$, instance 1) is a degraded contraction — its energy is 8 above that instance's best and it fails to return even the exact mirror — not a second optimum. The square marker asks the same question across contraction orders instead: one state from each of the eight lattice transformations, $\beta = 4$, median over five instances of the pairwise minimum, median and maximum. Eight states from eight orders span the configuration space; 1024 states from one order do not.

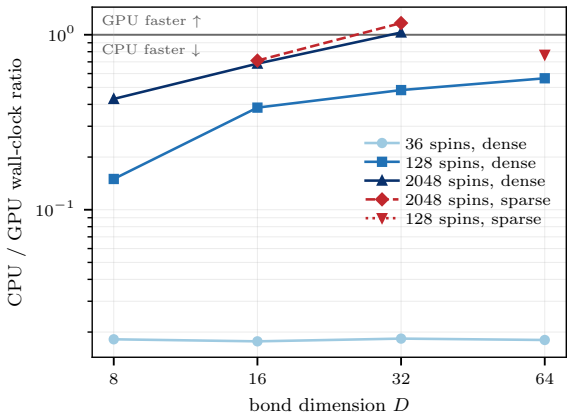

Figure 2: Where GPU execution begins to pay: CPU/GPU wall-clock ratio on identical configurations, so values below one mean the host is faster. The device wins only once both the instance and the bond dimension are large. Data as in Table 2.

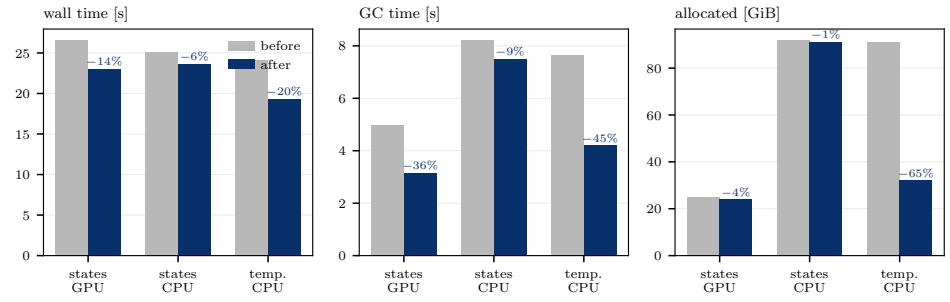

Figure 3: What the two allocation changes bought, on a 2048-spin bond-32 solve; percentages are the reduction. "states" is the matrix-backed branched configuration set, "temp." the off-heap contraction temporaries. Note the middle and right panels together: the states change cuts allocated *bytes* by only 4% on GPU yet garbage collection by 36%, because collection cost tracks the number of live objects rather than their volume.